DevMentor AI - By Dhruv Shukla

12 Aug 2026

Daily Learning Report

TODAY'S LEARNING

Inheritance

Day 6 | Intermediate | 10 min read

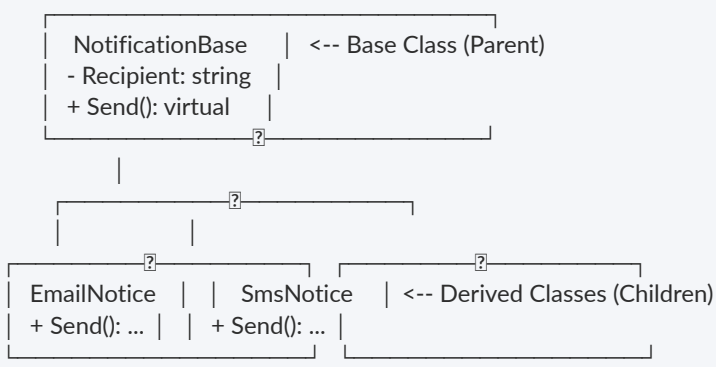
Generated by DevMentor AI by Dhruv Shukla

? Today's Goal

Dhruv will master C# object inheritance by understanding class hierarchies, constructor chaining with base, and method customization using virtual and override. By the end of this module, he will be able to design maintainable base classes, prevent common runtime execution bugs, and apply modern object-oriented principles in production ASP.NET Core applications.

? Core Concept

Inheritance allows a derived class (child) to inherit state (fields, properties) and behavior (methods) from a base class (parent). It solves code duplication across domain models that share common characteristics, enforcing a strict "is-a" relationship.



Internal Mechanism

- Memory Layout: When you instantiate a derived class, the .NET Common Language Runtime (CLR) allocates

a single contiguous memory block containing fields from the base class followed by fields of the derived class.

- Virtual Method Table (vtable): Mark a method as virtual, and the CLR creates a vtable entry for the class. Derived classes that specify override update this pointer, enabling dynamic dispatch (runtime selection of the correct method implementation).
- Constructor Execution: Base class constructors execute first, moving top-down from parent to child, ensuring the base state is fully initialized before child initialization runs.

Key Rules & Terminologies

- Single Inheritance: C# allows inheriting from only one base class.
- protected: Accessible inside the defining class and any derived class, but hidden from external consumers.
- base: Refers to parent class members and constructors (base()).
- sealed: Prevents a class from being inherited further.

What Happens If Done Wrong

Overusing inheritance creates tight coupling across deep hierarchies (fragile base class problem). Altering a base class method can silently break derived subclasses throughout the application.

```
using System;

public class NotificationBase
{
    public string Recipient { get; }

    public NotificationBase(string recipient)
    {
        Recipient = recipient ?? throw new ArgumentNullException(nameof(recipient));
    }

    public virtual void Send(string message)
    {
        Console.WriteLine($"[LOG]: Dispatching default notification to {Recipient}: {message}");
    }
}

public class EmailNotification : NotificationBase
{
    public string Subject { get; }

    // Constructor chaining to base class constructor
    public EmailNotification(string recipient, string subject) : base(recipient)
    {
        Subject = subject;
    }
```

```
    public override void Send(string message)
    {
        base.Send(message); // Retain base logging behavior
        Console.WriteLine($"[EMAIL]: Sent Subject '{Subject}' to {Recipient}");
    }
}

public class Program
{
    public static void Main()
    {
        NotificationBase notice = new EmailNotification("dhruv@example.com", "System Update");
        notice.Send("Server reboot at midnight.");
    }
}
```

? Real Project Example

In enterprise ASP.NET Core applications, base classes encapsulate common operational infrastructure—such as correlation logging, exception handling, and metrics tracking—across business service implementations.

```
using Microsoft.Extensions.Logging;
using System;
using System.Threading.Tasks;

namespace PaymentSystem.Services
{
    public abstract class BasePaymentProcessor
    {
        protected readonly ILogger<BasePaymentProcessor> Logger;

        protected BasePaymentProcessor(ILogger<BasePaymentProcessor> logger)
        {
            Logger = logger ?? throw new ArgumentNullException(nameof(logger));
        }

        public async Task<bool> ProcessTransactionAsync(decimal amount, string currency)
        {
            Logger.LogInformation("Initiating transaction of {Amount} {Currency}", amount, currency);
            try
            {
                return await ExecutePaymentLogicAsync(amount, currency);
            }
            catch (Exception ex)
```

```
        {
            Logger.LogError(ex, "Transaction failed for amount {Amount}", amount);
            return false;
        }
    }

    // Must be implemented by vendor-specific payment engines
    protected abstract Task<bool> ExecutePaymentLogicAsync(decimal amount, string currency);
}

public class StripePaymentProcessor : BasePaymentProcessor
{
    public StripePaymentProcessor(ILogger<BasePaymentProcessor> logger) : base(logger) { }

    protected override async Task<bool> ExecutePaymentLogicAsync(decimal amount, string currency)
    {
        // Simulated Stripe API call
        await Task.Delay(100);
        Logger.LogInformation("Stripe API charged {Amount} {Currency} successfully.", amount, currency);
        return true;
    }
}
}
```

Architectural Breakdown

- Template Method Pattern: ProcessTransactionAsync handles error boundaries and logging standardly across all gateways, while delegating actual payment execution to ExecutePaymentLogicAsync.
- Dependency Injection: Derived services receive singletons or scoped loggers via primary or standard constructors and pass them upstream via : base(logger).
- Senior Engineer Perspective: Prefer shallow inheritance trees (maximum 2 levels deep). When behavior variations become complex, favor composition (injecting strategies) over adding deeper child classes.

⚠ Top 3 Mistakes

1. Shadowing Methods using new Instead of override

Why it fails: Using new hides the base method instead of overriding it in the vtable. When cast to the base type, runtime polymorphism fails and calls the parent method instead.

```
// ⚠ BAD: Hides base method, breaking polymorphism
public class BaseService { public void Log() => Console.WriteLine("Base"); }
```

```
public class ChildService : BaseService { public new void Log() => Console.WriteLine("Child"); }

// Usage:
BaseService s = new ChildService();
s.Log(); // Output: "Base" (Unexpected bug!)
```

```
// ✅ GOOD: Proper dynamic method override
public class BaseService { public virtual void Log() => Console.WriteLine("Base"); }
public class ChildService : BaseService { public override void Log() => Console.WriteLine("Child"); }

// Usage:
BaseService s = new ChildService();
s.Log(); // Output: "Child" (Polymorphic call succeeds)
```

---

2. Calling Virtual Methods Inside Base Constructors

Why it fails: Base constructors run before derived constructors. Calling a virtual method in a base constructor invokes the derived override before derived fields are initialized, leading to NullReferenceException.

```
// ❌ BAD: Virtual call on uninitialized child state
public class BaseUser
{
    public BaseUser() { Init(); } // Calls overridden method early!
    public virtual void Init() { }
}

public class LeadUser : BaseUser
{
    private string _config;
    public LeadUser() { _config = "LOADED"; }
    public override void Init() => Console.WriteLine(_config.Length); // NullReferenceException!
}
```

```
// ✅ GOOD: Explicit initialization pattern after full construction
public class BaseUser
{
    public void Initialize() => OnInitialize();
    protected virtual void OnInitialize() { }
}

public class LeadUser : BaseUser
{
```

```
    private string _config = "LOADED"; // Direct initialization or complete inside constructor
    protected override void OnInitialize() => Console.WriteLine(_config.Length);
}
```

---

3. Creating Deep Inheritance Trees (Over-Inheriting)

Why it fails: Creating deep class chains (Entity -> NamedEntity -> AuditableEntity -> User -> AdminUser) causes brittle code where small base changes break every child level unexpectedly.

```
// ❌ BAD: Multi-layered deeply coupled class hierarchy
public class Entity { public int Id { get; set; } }
public class AuditableEntity : Entity { public DateTime Created { get; set; } }
public class PersonEntity : AuditableEntity { public string Name { get; set; } }
public class EmployeeEntity : PersonEntity { public decimal Salary { get; set; } }
```

```
// ✅ GOOD: Prefer shallow inheritance + flat composition interface design
public interface IAuditable { DateTime Created { get; set; } }

public class Employee : IAuditable
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Salary { get; set; }
    public DateTime Created { get; set; }
}
```

🗞️ Industry News

- Azure Developer CLI extension framework is GA: build dev workflows for apps using Azure

Microsoft announced the General Availability of the Azure Developer CLI (azd) extension framework. This allows teams to build custom workflow steps directly into Azure deployment pipelines. For developers, writing modular, reusable components in ecosystem tools requires understanding clean OOP contracts and architectural patterns.

🔗 Read full article

- From coder to orchestrator: How agents shift the role of a developer

GitHub highlights how autonomous AI agents are shifting developer duties from manual coding to

orchestrating architecture and code quality. Dhruv must master structural fundamentals like object-oriented design and typing rules to review and govern generated code effectively.

Read full article

- .NET 11 Preview 7 is now available!

Microsoft released .NET 11 Preview 7, bringing early optimizations to runtime typing systems and performance tweaks for virtual method lookups. Staying updated with language release cadences ensures developers leverage modern runtime enhancements and low-overhead OOP executions.

Read full article

- .NET and .NET Framework August 2026 servicing releases updates

Microsoft released security updates addressing critical runtime memory vulnerabilities across current .NET servicing releases. Understanding memory allocations in class instances and base type initialization structures ensures developers write code secure against runtime state corruptions.

Read full article

- Today I will... manage Git Submodules without leaving the IDE

Visual Studio introduced native workspace tooling to manage nested Git submodules directly within the IDE UI. Modern component architecture relies heavily on clean separation, both in project structures and OOP class abstractions.

Read full article

- How Netflix Scaled Its Real-Time Service Map

Netflix documented scaling their real-time distributed service maps to handle high-throughput telemetry updates. Robust backend services rely on foundational C# state inheritance and interface contracts to process domain events uniformly under heavy workloads.

Read full article

- IBM and Red Hat Expand Lightwell to Strengthen Trust and Governance for AI-Era Open Source

IBM and Red Hat extended project Lightwell to govern trust, compliance, and component licensing across AI development frameworks. Explicit type hierarchies and base class abstractions play a key role in building well-defined enterprise boundaries for software compliance.

Read full article

Interview Questions & Answers

Q1: What is the difference between protected and internal access modifiers in C#?

A1: protected members are accessible only within their declaring class and any class that derives from it, regardless of the assembly. internal members are accessible anywhere within the same assembly, but hidden from external assemblies. You can combine them as protected internal, allowing access to both derived classes and any class within the same assembly.

```
public class Parent
{
    protected int ProtectedField; // Accessible to subclasses
    internal int InternalField;   // Accessible within assembly
}
```

Q2: Why doesn't C# support multiple class inheritance, and how do we achieve similar behavior?

A2: C# disallows multiple class inheritance to eliminate complexity and ambiguity, such as the Diamond Problem (where two base classes implement the same method differently). Instead, C# supports multiple interface implementation, allowing a class to adhere to multiple behavioral contracts while deriving state from at most one base class.

Q3: What is the difference between override and new (shadowing) keywords when inheriting a method?

A3: override replaces the base class vtable entry, enabling polymorphic runtime resolution even when referenced via a base class variable. The new modifier explicitly hides the inherited base method, creating a separate unlinked method that bypasses runtime dynamic dispatch.

```
BaseClass obj = new DerivedClass();
obj.VirtualMethod(); // Override calls Derived implementation; 'new' calls Base implementation.
```

Q4: How does constructor execution order work in an inheritance hierarchy?

A4: Constructors execute in a top-down chain starting from the root base class down to the derived class. The CLR ensures base class state is fully initialized via implicit or explicit : base(...) invocation before executing the derived constructor body.

Q5: How does the .NET runtime (CLR) resolve virtual method calls internally?

A5: The CLR uses a Virtual Method Table (vtable) per type containing function pointers for virtual methods.

When calling a virtual method, the CLR checks the actual object type in memory, looks up its vtable pointer, and dynamically dispatches execution to the method address stored at runtime.

Q6: What is the Fragile Base Class problem, and how can C# language design mitigate it?

A6: The Fragile Base Class problem occurs when modifications to a base class unintentionally break derived subclasses. C# mitigates this by requiring explicit virtual declarations on base methods and explicit override keywords on derived methods, preventing accidental method overrides. Developers can also mark classes as sealed to prevent unsafe inheritance entirely.

```
public sealed class SecurityTokenProvider // Prevents subclass fragile coupling
{
}
```

? Revision Summary

Day 5: Classes and Objects

Classes define the blueprints for reference types, binding data (fields, properties) and behavior (methods) into an encapsulated unit instantiated on the heap.

- Key Takeaway: Objects are heap-allocated instances managed by GC; proper encapsulation prevents invalid object states.

Day 3: Control Statements

Control statements (if, switch, foreach, while) direct code execution flows based on dynamic runtime logic and evaluation conditions.

- Key Takeaway: Choose pattern matching switch expressions over deeply nested if-else blocks to keep code clean and readable.

? Tomorrow Preview

Tomorrow we step up to Polymorphism and Abstract Classes/Interfaces. You'll learn how to write decoupled systems that operate against pure behavioral contracts, replacing explicit subclass branching with dynamic, pluggable enterprise application components.